



Machinelearning Steps



Max Kleiner 7 min read · 1 day ago



2



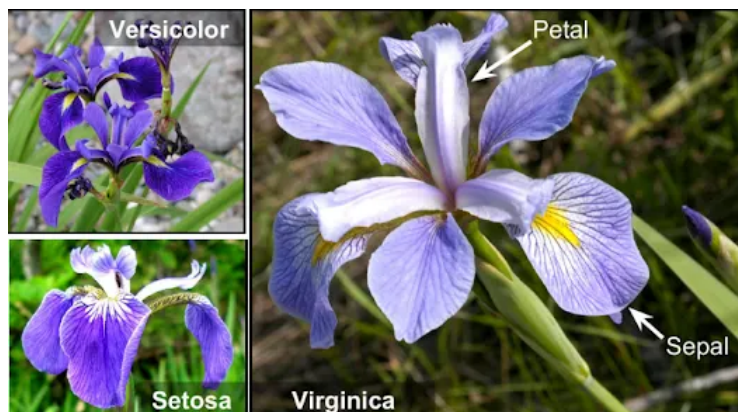
1



This article will provide the clear cut understanding of Iris dataset and how to do classification on Iris flowers dataset using python, seaborn and sklearn.

The Iris dataset is often used in machine learning and data science courses, because it's simple to understand and well-defined, yet interesting enough to present real challenges to new learners. This **single code snippet** will use Python with Python4Delphi to classify the Iris dataset into one of three flower species: Setosa, Versicolor, or Virginica.

The iris data consisted of 150 samples of three species of Iris. The first column represented sepal length, the second column represented sepal width, the third column represented petal length, and the fourth column represented petal width. I'm going to use sci-kit-learn to classify these instances according to their species.



The provided Delphi code snippet uses the Python4Delphi library, specifically the TPythonEngine component, to embed Python code execution into a Delphi application. It creates an instance of TPythonEngine and uses it to run a sequence of Python commands that implement a machine learning workflow with the iris dataset using popular Python libraries like pandas, matplotlib, pyplot, scikit-learn (sklearn), and seaborn.

Import Libraries

First we load the libraries:

```
with TPythonEngine.Create(Nil) do begin //pythonhome:= 'C:\Users\User\AppData\Lo
```

```
with TPythonEngine.Create(Nil) do begin
    //pythonhome:= 'C:\Users\User\AppData\Local\Programs\Python\Python314\';
    try
        loadDLL;
        autofinalize:= false;
        ExecStr('import pandas as pd'+lf+
            'import matplotlib.pyplot as plt');
        execStr('from sklearn.model_selection import train_test_split'+lf+
            'from sklearn.ensemble import RandomForestClassifier'+lf+
            'from sklearn import datasets'+lf+
            'from sklearn.metrics import accuracy_score'+lf+
            'from sklearn.metrics import confusion_matrix');
```

Load Dataset

But the data are not well structured for us to understand. So we need to convert it into a pandas DataFrame. Pandas is a great tool for doing all sorts of things related to datasets, including preprocessing and exploring them.

```
execstr('iris = datasets.load_iris()+lf+ 'df = pd.DataFrame(iris.data, columns=
```

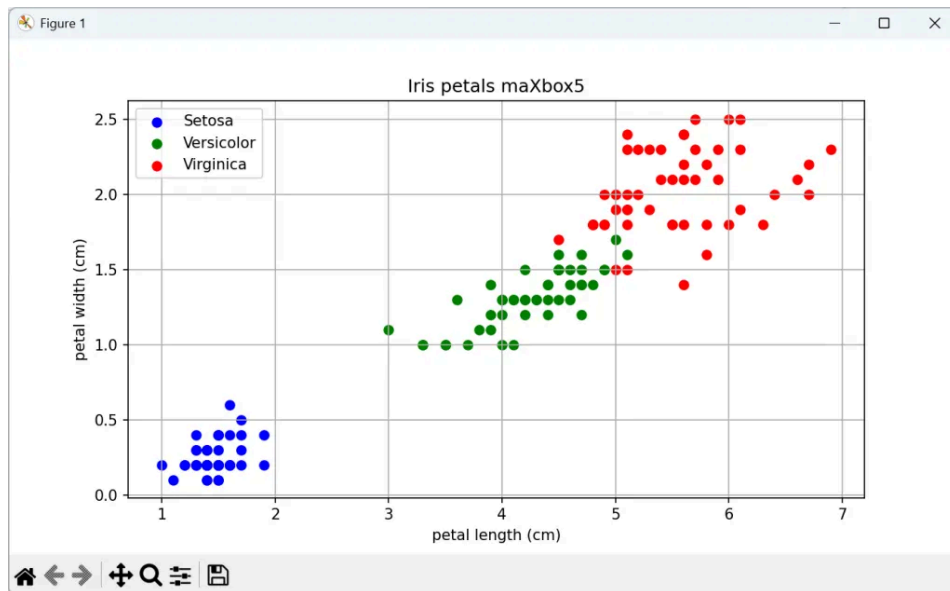
```
execstr('iris = datasets.load_iris()+lf+
        'df = pd.DataFrame(iris.data, columns=iris.feature_names)');
```

- It loads the iris dataset from sklearn.datasets and converts it into a pandas DataFrame.
- The data is split into training and test sets with stratification and fixed random seed as random_state for reproducibility and comparison.

Explore Dataset

The dataset contains **150 samples** of iris flowers, divided into three species: **Setosa**, **Versicolor**, and **Virginica**. Each sample is described by **four features** measured in centimeters:

- **Sepal Length:** The length of the flower's sepal.
- **Sepal Width:** The width of the flower's sepal.
- **Petal Length:** The length of the flower's petal.
- **Petal Width:** The width of the flower's petal.



```
# labels and scatter points fig, ax = plt.subplots()
```

Classify the species

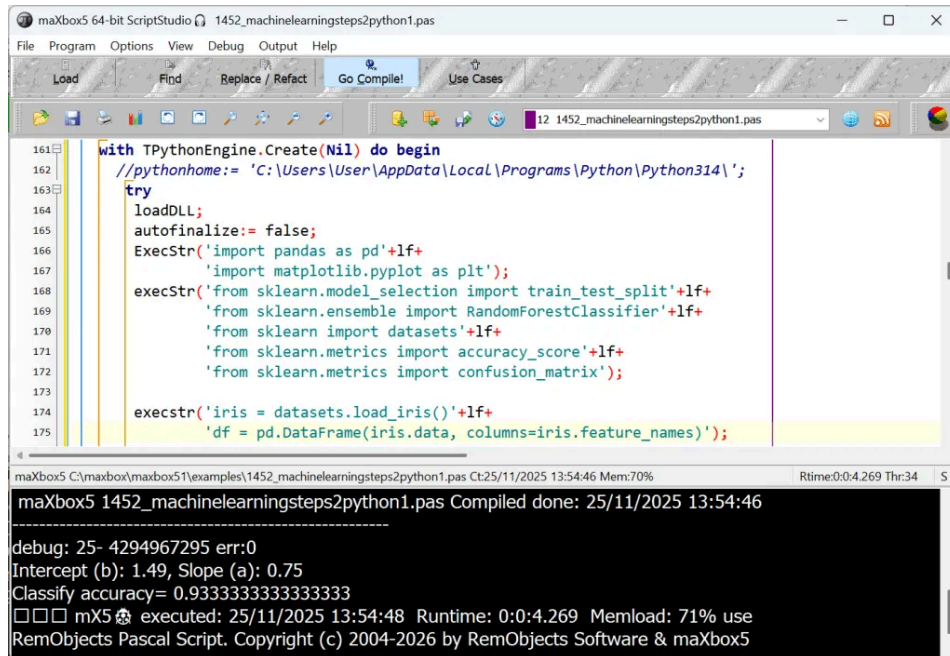
So we don't need to hardcode the algorithm for classification since there are already many algorithms available in the sci-kit learn package. We can simply choose any of them and use them. Here, I am going to use the Random Forest Classifier model. Now, after training our model on training data, we can predict petal and sepal measurements on testing data as the features. And that's the non binary cassification for the first block!

```
execstr('X_train,X_test, y_train,y_test = train_test_split(df[iris.feature_names
```

```
execstr('X_train,X_test, y_train,y_test = train_test_split(df[iris.feature_names
            'iris.target,test_size=0.5, stratify=iris.target, random_state=123456)')
execstr('rf = RandomForestClassifier(n_estimators=100, oob_score=True, '+
            'random_state=123456)'+lf+'rf.fit(X_train,
writeLn('Classify accuracy= '+Evalstr('accuracy_score(y_test, rf.predict(X_test)
```

Also, we can use the **train_test_split** function in sklearn in order to split the dataset into train and test.

We get Classify accuracy= 0.9333333333333333



```

161 with TPythonEngine.Create(Nil) do begin
162   //pythonhome:= 'C:\Users\User\AppData\Local\Programs\Python\Python314\';
163   try
164     loadDLL;
165     autofinalize:= false;
166     ExecStr('import pandas as pd'+lf+
167             'import matplotlib.pyplot as plt');
168     execStr('from sklearn.model_selection import train_test_split'+lf+
169            'from sklearn.ensemble import RandomForestClassifier'+lf+
170            'from sklearn import datasets'+lf+
171            'from sklearn.metrics import accuracy_score'+lf+
172            'from sklearn.metrics import confusion_matrix');
173
174     execstr('iris = datasets.load_iris()'+lf+
175            'df = pd.DataFrame(iris.data, columns=iris.feature_names)');

```

maXbox5 C:\maxbox\maxbox51\examples\1452_machinelearningsteps2python1.pas Ct:25/11/2025 13:54:46 Mem:70% Rtime:0:0:4.269 Thr:34 S

maXbox5 1452_machinelearningsteps2python1.pas Compiled done: 25/11/2025 13:54:46

debug: 25- 4294967295 err:0

Intercept (b): 1.49, Slope (a): 0.75

Classify accuracy= 0.9333333333333333

□□□ mX5 executed: 25/11/2025 13:54:48 Runtime: 0:0:4.269 Memload: 71% use

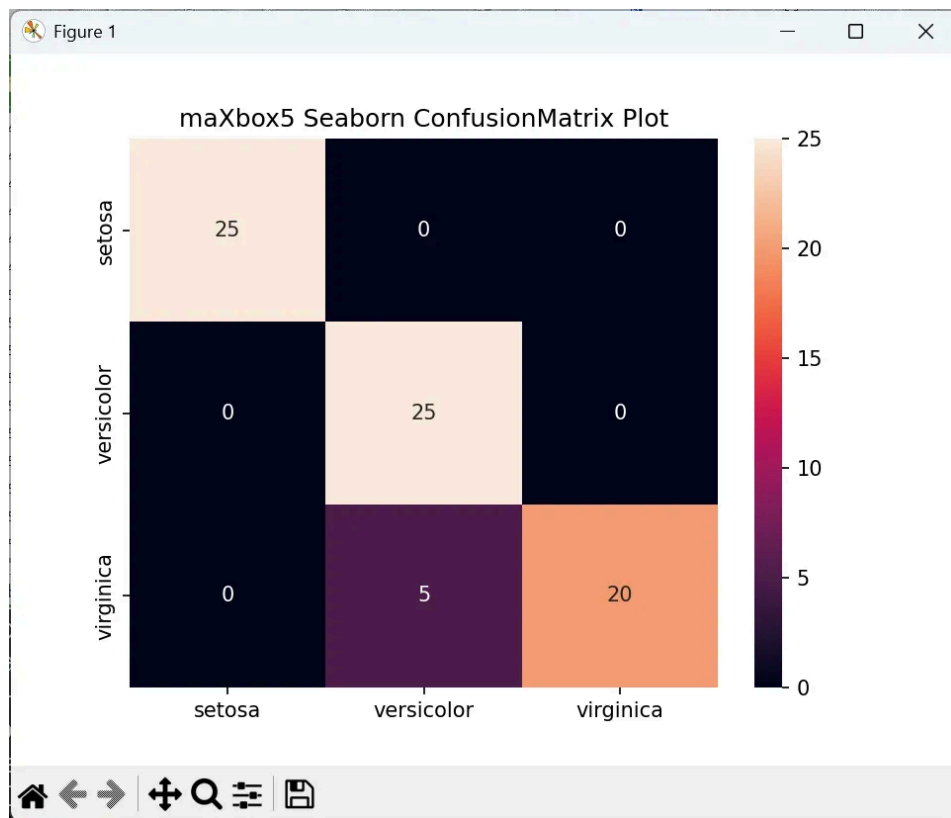
RemObjects Pascal Script. Copyright (c) 2004-2026 by RemObjects Software & maXbox5

https://sourceforge.net/projects/maxbox5/files/examples/1452_machinelearningsteps2python1.pas/download

Plotting the Performance Measures

Performance measures are used to evaluate effectiveness of classifiers on different datasets with different characteristics. For classification problems, there are three main measures for evaluating the model, the **precision** (the accuracy of positive predictions or the number of most relevant values from retrieved values.), **Recall** (ratio of positive instances that are truly detected by the classifier), and **confusion matrix**.

In other words if you don't recognize a person then its bad recall and if you recognize a person for another (confuse with) then its bad precision!



Confusion Matrix for Performance Measure

The scores are pretty good in this case only 5 misclassified. The main usage of the confusion matrix is to identify how many of the classes are misclassified by the classifier (false-positive or false-negative).

Access to all of it.

Become a Medium member for unlimited reading.

Upgrade now

With the following call you get a report of the performance:

```
execstr('print(metrics.classification_report(y_test, rf.predict(X_test), digits=3))');
```

precision recall f1-score support eg: Setosa(0), Versicolor(1), Virginica(2)

```
precision recall f1-score support 0 1.000 1.000 1.000 25 1 0.833 1.000 0.909 25
```

precision recall f1-score support eg: Setosa(0), Versicolor(1), Virginica(2)

		precision	recall	f1-score	support
1					
2	0	1.000	1.000	1.000	25
3	1	0.833	1.000	0.909	25
4	2	1.000	0.800	0.889	25
5					
6	accuracy			0.933	75

macro avg 0.944 0.933 0.933 75

weighted avg 0.944 0.933 0.933 75

We check the **recall** of virginica (2) its 80%, that means only 80% are recognized (**recall**) as virginica so 20% are misclassified as versicolor!

On the other side the **precision** of versicolor (1) is only 83% because 5 from virginica are misclassified as versicolor and that means $25/30 = 0.8333$ or 83% precision!

Conclusion and Summary

The provided Delphi code snippet uses the Python4Delphi library, specifically the TPythonEngine component, to embed Python code execution into a Delphi application. It creates an instance of TPythonEngine and uses it to run a sequence of Python commands that implement a machine learning workflow with the iris dataset using popular Python libraries like pandas, matplotlib, scikit-learn, and seaborn.

Here are the detailed steps in 9 points and purpose of the code for your tests and document:

- It initializes the Python engine component (TPythonEngine.Create(nil)) and loads the Python DLL dynamically (loadDLL) as 3.13 or 3.14.
- The code sets autofinalize to false to control Python finalization manually.
- It executes multiple Python import statements to load common data science and machine learning libraries: pandas (for data manipulation), matplotlib.pyplot (plotting), sklearn's modules including train_test_split, RandomForestClassifier, dataset loaders, and metrics like accuracy_score and confusion_matrix and It loads the iris dataset from sklearn.datasets and converts it into a pandas DataFrame.
- The data is split into training and test sets with stratification and fixed random seed for reproducibility.
- A RandomForestClassifier model is created and trained on the training data.

- The accuracy of the model on the test set is evaluated and printed using the Python `accuracy_score` function and a confusion matrix is generated, converted into a pandas DataFrame, and plotted using seaborn's heatmap with an annotated matrix plot.
- The plot is finalized with a title and displayed using matplotlib.
- Exception handling surrounds the block to raise any Python-related errors caught during execution.
- Finally, resources are freed explicitly by calling `Free` on the `TPythonEngine` instance.

This code showcases an integration approach where Delphi acts as a container to drive Python-based data science workflows, combining the strengths of Delphi in application development and Python in machine learning and visualization. The use of the embedded Python environment allows direct execution of Python commands from Delphi, enabling rich interaction between the two languages.

```
procedure PY_Machinelearning_steps; begin with TPythonEngine.Create(Nil) do begi
```

```
procedure PY_Machinelearning_steps;
begin
  with TPythonEngine.Create(Nil) do begin
    //pythonhome:= 'C:\Users\User\AppData\Local\Programs\Python\Python314\';
    try
      loadDLL;
      autofinalize:= false;
      ExecStr('import pandas as pd'+lf+
        'import matplotlib.pyplot as plt');
      execStr('from sklearn.model_selection import train_test_split'+lf+
        'from sklearn.ensemble import RandomForestClassifier'+lf+
        'from sklearn import datasets'+lf+
        'from sklearn.metrics import accuracy_score'+lf+
        'from sklearn.metrics import confusion_matrix');

      execstr('iris = datasets.load_iris()'+lf+
        'df = pd.DataFrame(iris.data, columns=iris.feature_names)');
      execstr('X_train,X_test, y_train,y_test= train_test_split(df[iris.feature_n
        'iris.target,test_size=0.5, stratify=iris.target, random_state=1234
      execstr('rf = RandomForestClassifier(n_estimators=100, oob_score=True, '+
        'random_state=123456)'+lf+
        'rf.fit(X_train, y_train)');

      writeln('Classify accuracy= ' +evalstr('accuracy_score(y_test,rf.predict(X_t

//https://www.blopig.com/blog/2017/07/using-random-forests-in-python-with-scikit
      execstr('import seaborn as sns');
      execstr('cm = pd.DataFrame(confusion_matrix(y_test, rf.predict(X_test)),'+
        'columns=iris.target_names, index=iris.target_names)');
      execstr('sns.heatmap(cm, annot=True)'+lf+
        'plt.title("maXbox5 Seaborn ConfusionMatrix Plot")'+lf+
        'plt.show()');
    except
      raiseError;
    finally
      Free;
  end;
end;
```

```
end;
end;
```

```
1 procedure PY_Machinelearning_steps;
2 begin
3   with TPythonEngine.Create(nil) do begin
4     //pythonhome:= 'C:\Users\User\AppData\Local\Programs\Python\Python314\'
5     try
6       loadDLL;
7       autofinalize:= false;
8       ExecStr('import pandas as pd'+lf+
9             'import matplotlib.pyplot as plt');
10      execStr('from sklearn.model_selection import train_test_split'+lf+
11            'from sklearn.ensemble import RandomForestClassifier'+lf+
```

Post Reader ? Howdy, maxkleiner

plan to remove the banner and unlock more features, from US\$4/month

Upgrade

```
16   execstr('iris = datasets.load_iris()'+lf+
17         'df = pd.DataFrame(iris.data, columns=iris.feature_names)');
18   execstr('X_train,X_test, y_train,y_test= train_test_split(df[iris.featu
19         'iris.target,test_size=0.5, stratify=iris.target, random_state=
20   execstr('rf = RandomForestClassifier(n_estimators=100, oob_score=True,
21         'random_state=123456)'+lf+
22         'rf.fit(X_train, y_train)');
23
24   writeln('Classify accuracy= '+evalstr('accuracy_score(y_test,rf.predict
25
26   //https://www.blopig.com/blog/2017/07/using-random-forests-in-python-with-sc
27   execstr('import seaborn as sns');
28   execstr('cm = pd.DataFrame(confusion_matrix(y_test, rf.predict(X_test))
29         'columns=iris.target_names, index=iris.target_names)');
30   execstr('sns.heatmap(cm, annot=True)'+lf+
31         'plt.title("maXbox5 Seaborn ConfusionMatrix Plot")'+lf+
32         'plt.show()');
33   except
34     raiseError;
35   finally
36     Free;
37   end;
38 end;
39 end;
```

References:

- TPythonEngine is a core component of Python4Delphi for embedding Python environment.
- The code sequence corresponds to classical iris dataset classification using Random Forest, a well-known machine learning example.
- maXbox5 as the scripting-studio compiler to execute code.

This code snippet can form a comprehensive description for your document highlighting how Delphi and Python4Delphi facilitate machine learning integration through Python scripts executed inside Delphi or maXbox applications. So with maXbox you we can think out of the box.



Classify 3 Locs as our next step



Classify with 4 Classes : Swiss, France, Germany and Belgian Locs

Update 1.2

The update add **classification_report** and data explorer image scatter plot.

[Download 1452_machinelearningsteps2python12plotdataset.pas \(maXbox5\).](#)

```

161 procedure PY_Machinelearning_steps;
162 begin
163   with TPythonEngine.Create(Nil) do begin
164     //pythonhome:= 'C:\Users\User\AppData\Local\Programs\Python\Python314\';
165     try
166       loadDLL;
167       autofinalize:= false;
168       ExecStr('import pandas as pd, io, sys'+lf+
169         'import matplotlib.pyplot as plt');
170       execStr('from sklearn.model_selection import train_test_split'+lf+
171         'from sklearn.ensemble import RandomForestClassifier'+lf+
172         'from sklearn import datasets'+lf+
173         'from sklearn.metrics import accuracy_score'+lf+
174         'from sklearn.metrics import confusion_matrix');
175       execStr('output = io.StringIO(); sys.stdout = output');

```

maXbox5 C:\maXbox5\maXbox5\examples\1452_machinelearningsteps2python12.pas Ct:26/11/2025 14:03:24 Mem:66% Rtime:0:0:5.1 Thr:36 S

```

Intercept (b): 1.49, Slope (a): 0.75
Classify accuracy= 0.9333333333333333
Precision, Recall, Confusion matrix, in testing

```

	precision	recall	f1-score	support
0	1.000	1.000	1.000	25
1	0.833	1.000	0.909	25
2	1.000	0.800	0.889	25
accuracy			0.933	75
macro avg	0.944	0.933	0.933	75
weighted avg	0.944	0.933	0.933	75

mX5 executed: 26/11/2025 14:03:26 Runtime: 0:0:5.1 Memload: 66% use
RemObjects Pascal Script. Copyright (c) 2004-2026 by RemObjects Software & maXbox5

[Download 1452_machinelearningsteps2python12.pas \(maXbox5\)](#)

. . .

Originally published at <http://maxbox6.wordpress.com> on November 25, 2025.

Machine Learning

Machine Learning Ai

Classification

Image Classification

Python Programming

**Written by Max Kleiner**

41 followers · 3 following

[Edit profile](#)

Max Kleiner's professional environment is in the areas of OOP, UML and coding - among other things as a trainer, developer and consultant.

Responses (1)



Max Kleiner

What are your thoughts?



Breitsch

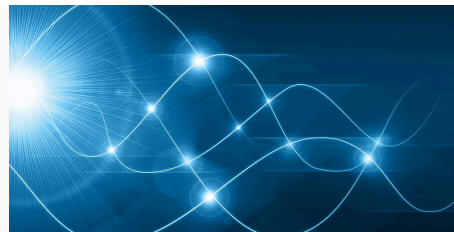
4 hours ago



Usefull article so I can also see now the tradeoff between precision (false-positive) and recall (false-negative) in a classification.

[Reply](#)

More from Max Kleiner



 In Analytics Vidhya by Leland Roberts

Understanding the Mel Spectrogram

(and Other Topics in Signal Processing)

W⁺ ..

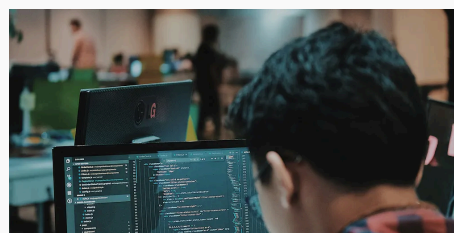


 Max Kleiner

Python for Delphi (P4D) is a set of free components that wrap up the Python DLL...

 W^+ •

Recommended from Medium



 In Start with Data Engineering by Shaloo Mathew

Learn Database Management System from Scratch- Part 2

In 1970, Edgar F. Codd transformed data management with a revolutionary idea:...

3d ago



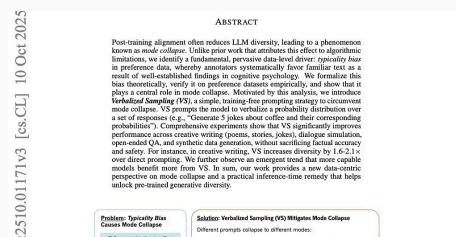
Sep 28



52



1



In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I...

ChatGPT keeps giving you the same boring response? This new technique unlocks 2x...



Oct 20



18.3K



443



In Science Spectrum by Cole Frederick

How to Find Pi in the Woods

Using Math Creatively



3d ago



446



8



In Level Up Coding by Fareed Khan

Building the 14 Key Pillars of Agentic AI

Parallelism, Ensembles, Retrieval and more



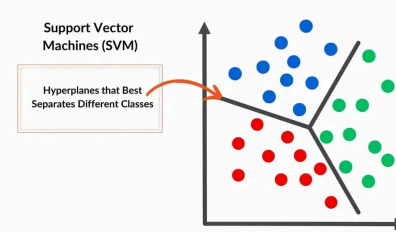
5d ago



792



5



Hemasri

Support Vector Machine (SVM)

Part 5: Learning Machine Learning/Deep Learning

See more recommendations

maXbox4exe

Digitally signed by

maXbox4exe

Date: 2025.11.26 21:54:11

+01'00'